FULL-STACK TAKE-HOME ASSIGNMENT

React + C# / ASP.NET Core | Junior Developer

Role	Junior Full-Stack Developer (React + C# / .NET)
Experience Level	6–12 months
Time Limit	6–8 hours (no more)
Submission	GitHub repository link with a README

The Task: Build a "Project Task Board" Application

Build a task management application where users can organize tasks under projects, track progress, and add comments to tasks. The app must have a React frontend and a C# / ASP.NET Core Web API backend with data persisted in a SQLite database using EF Core with proper migrations.

Data Model

Project

Field	Type	Rules
Id	int	Auto-generated primary key
Name	string	Required, max 100 chars, unique
Description	string	Optional, max 300 chars
CreatedAt	DateTime	Set automatically on creation

Task

Field	Type	Rules
Id	int	Auto-generated primary key
ProjectId	int	Required — foreign key to Project
Title	string	Required, max 150 chars
Description	string	Optional, max 1000 chars
Priority	enum	Low, Medium, High, Critical
Status	enum	Todo, InProgress, Review, Done
DueDate	DateTime	Optional, must be today or future when created
CreatedAt	DateTime	Set automatically on creation
UpdatedAt	DateTime	Set automatically on every update

Comment

Field	Type	Rules
Id	int	Auto-generated primary key
TaskId	int	Required — foreign key to Task
Author	string	Required, max 50 chars
Body	string	Required, max 500 chars
CreatedAt	DateTime	Set automatically on creation

Relationships: A Project has many Tasks. A Task belongs to one Project. A Task has many Comments. Deleting a Project should cascade-delete its Tasks (and their Comments). Deleting a Task should cascade-delete its Comments.

API Endpoints

Projects

Method	Endpoint	Description
GET	/api/projects	List all projects with task count per status
POST	/api/projects	Create a new project
GET	/api/projects/{id}	Get project details including its tasks
PUT	/api/projects/{id}	Update a project
DELETE	/api/projects/{id}	Delete a project and all associated data

Tasks

Method	Endpoint	Description
GET	/api/projects/{projectId}/tasks	List tasks (filtering, sorting, pagination)
POST	/api/projects/{projectId}/tasks	Create a task under a project
GET	/api/tasks/{id}	Get a single task with its comments
PUT	/api/tasks/{id}	Update a task
DELETE	/api/tasks/{id}	Delete a task and its comments

Comments

Method	Endpoint	Description
GET	/api/tasks/{taskId}/comments	List comments for a task
POST	/api/tasks/{taskId}/comments	Add a comment to a task
DELETE	/api/comments/{id}	Delete a comment

Dashboard

Method	Endpoint	Description
GET	/api/dashboard	Summary: total projects, tasks by status, overdue count, due within 7 days

API Feature Requirements

Filtering & Sorting (GET tasks)

The task list endpoint must support these query parameters:

- **status** — filter by status (e.g. ?status=InProgress)

- **priority** — filter by priority (e.g. ?priority=High)
- **sortBy** — one of dueDate, priority, createdAt (default: createdAt)
- **sortDir** — asc or desc (default: desc)

Pagination (GET tasks)

- **page** — page number (default: 1)
- **pageSize** — items per page (default: 10, max: 50)

Response must include pagination metadata:

```
{ "data": [ ... ],
  "page": 1,
  "pageSize": 10,
  "totalCount": 47,
  "totalPages": 5 }
```

Validation & Error Handling

- Return proper HTTP status codes: 200, 201, 400, 404, 409 (duplicate project name), 500.
- Validation errors should return a structured JSON response with an errors object keyed by field name.
- Use a global exception handling middleware — unhandled exceptions should return a generic 500, never leak stack traces.

Frontend (React)

Required Views

1. Dashboard View — Landing page showing summary stats from /api/dashboard. Display total tasks, breakdown by status (simple bars or progress indicators), overdue count (highlighted in red), and upcoming due tasks.

2. Project List View — Show all projects as cards. Each card displays name, description, and a small status summary (e.g. "3 Todo · 2 In Progress · 1 Done"). Clicking navigates to the task board.

3. Task Board View (per project) — Display project tasks with filter dropdowns (status, priority), sort controls, and pagination. Each task shows title, priority badge (color-coded), status, due date (highlight if overdue). Clicking opens detail view.

4. Task Detail View — Full task information with an edit form. Below, a comments section with the ability to add new comments and delete existing ones.

5. Add/Edit Forms — Forms for creating and editing projects and tasks. Must show inline validation errors returned from the API.

Technical Requirements

- Use functional components and hooks: useState, useEffect, useContext.
- **Custom hook** useApi (or similar) that wraps fetch/axios and handles loading, error, and data states — used across all API calls.
- **React Context** to manage at least one piece of global state (e.g. selected project, dashboard data, or notifications).
- **React Router** with at least 3 routes: dashboard, project board, task detail.
- Handle and display loading spinners and error messages for all async operations.
- Show confirmation dialogs before any delete action.

Persistence & Database Requirements

This is a key evaluation area. The application **must** persist data across server restarts.

- Use SQLite as the database (file-based — no external DB server needed).
- Use EF Core as the ORM with the Microsoft.EntityFrameworkCore.Sqlite provider.
- Include at least one EF Core migration checked into the repo. README must include migration commands.
- Configure relationships, cascade deletes, and indexes in the DbContext using Fluent API or Data Annotations.
- **Seed data:** Populate the database with sample projects, tasks, and comments on first run so reviewers see data immediately.
- **Auto-timestamps:** CreatedAt and UpdatedAt must be set automatically via SaveChangesAsync override in DbContext — not manually in controllers or services.

Architecture & Code Organization

Backend — Expected Structure

```
TaskBoard.Api/
├── Controllers/       # Thin API controllers
├── Services/          # Business logic (interfaces + implementations)
├── Models/            # Entity classes
├── DTOs/              # Request and response DTOs
├── Data/
│   ├── AppDbContext.cs  # EF Core DbContext with Fluent API
│   └── SeedData.cs      # Database seeding logic
├── Middleware/        # Global exception handler
├── Migrations/        # EF Core migrations (checked in)
└── Program.cs
```

Key: Controllers must be thin (no direct DbContext usage). Use DTOs for API input/output, not raw entities. Register services via DI. Use async/await throughout.

Frontend — Expected Structure

```
task-board-ui/src/
├── components/   # Reusable UI (TaskCard, PriorityBadge, etc.)
├── pages/        # Route-level (Dashboard, ProjectBoard, TaskDetail)
├── hooks/        # Custom hooks (useApi, etc.)
├── context/      # React Context providers
├── services/     # API call functions
└── App.jsx       # Router setup
```

Key: API base URL must be configurable (env variable or config file), not hardcoded. No single 500-line component files.

Evaluation Criteria

Area	What We Look For	Weight
Working End-to-End	App runs, CRUD works, filtering/sorting/pagination functional	25%
Persistence & Data Layer	SQLite with migrations, seed data, relationships, cascades, auto-timestamps	20%
C# / API Design	REST conventions, status codes, DTOs, validation, error middleware, service layer, async	20%
React Quality	Custom hooks, Context, routing, component structure, loading/error handling, form validation	20%
Code Quality	Clean folder structure, naming, no dead code, separation of concerns	10%
README & Setup	Reviewer can clone, migrate, seed, and run in under 5 minutes	5%

> *We value **working, well-structured code** over feature count. A clean app that covers the core requirements will score higher than a feature-packed app with bugs.*

Bonus (Optional — Pick at Most Two)

These are not expected and won't count against anyone who skips them. They're an opportunity to show extra depth.

- **Optimistic UI updates:** Update React state immediately on mutations and roll back on failure.
- **Debounced search:** Add a search bar that searches by title with a debounced API call.
- **Audit trail:** Track status changes in a TaskHistory table (old → new status, timestamp). Display history in task detail.
- **Backend unit tests:** Write 3–5 xUnit tests for the Task service layer, mocking the DbContext.
- **Drag-and-drop:** Allow dragging task cards between status columns (Kanban-style).
- **Dark mode toggle:** Theme switcher using React Context, persisted in localStorage.

Submission Instructions

1. Push your code to a public or private GitHub repository.
2. Include a README.md at the root with: project description, prerequisites, how to apply migrations and seed the database, how to run backend and frontend, and any assumptions or design decisions.
3. **Commit history matters** — we want to see incremental progress, not a single giant commit. Work naturally.
4. Share the repository link by [INSERT DEADLINE].

If using a private repo, please add [INSERT GITHUB USERNAME] as a collaborator.

Starter References

Backend setup:

```
dotnet new webapi -n TaskBoard.Api
cd TaskBoard.Api
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
dotnet add package Microsoft.EntityFrameworkCore.Design
```

Create first migration:

```
dotnet ef migrations add InitialCreate
dotnet ef database update
```

Frontend setup:

```
npx create-react-app task-board-ui
cd task-board-ui
npm install react-router-dom axios
npm start
```

CORS in Program.cs:

```
builder.Services.AddCors(options => {
    options.AddDefaultPolicy(policy => {
        policy.AllowAnyOrigin().AllowAnyMethod().AllowAnyHeader();
    });
});
app.UseCors();
```

Questions?

If anything is unclear, email [INSERT CONTACT EMAIL]. Asking clarifying questions is always a positive signal.

Good luck — we look forward to reviewing your work!